

Version Control and Collaboration

Basics

Why Version Control? The Problem

Why Version Control? The Problem

- ▶ Files named:

Why Version Control? The Problem

- ▶ Files named:
 - ▶ `final.c`

Why Version Control? The Problem

- ▶ Files named:
 - ▶ `final.c`
 - ▶ `final_v2.c`

Why Version Control? The Problem

- ▶ Files named:
 - ▶ `final.c`
 - ▶ `final_v2.c`
 - ▶ `final_really_final.c`

Why Version Control? The Problem

- ▶ Files named:
 - ▶ `final.c`
 - ▶ `final_v2.c`
 - ▶ `final_really_final.c`
- ▶ No clear history

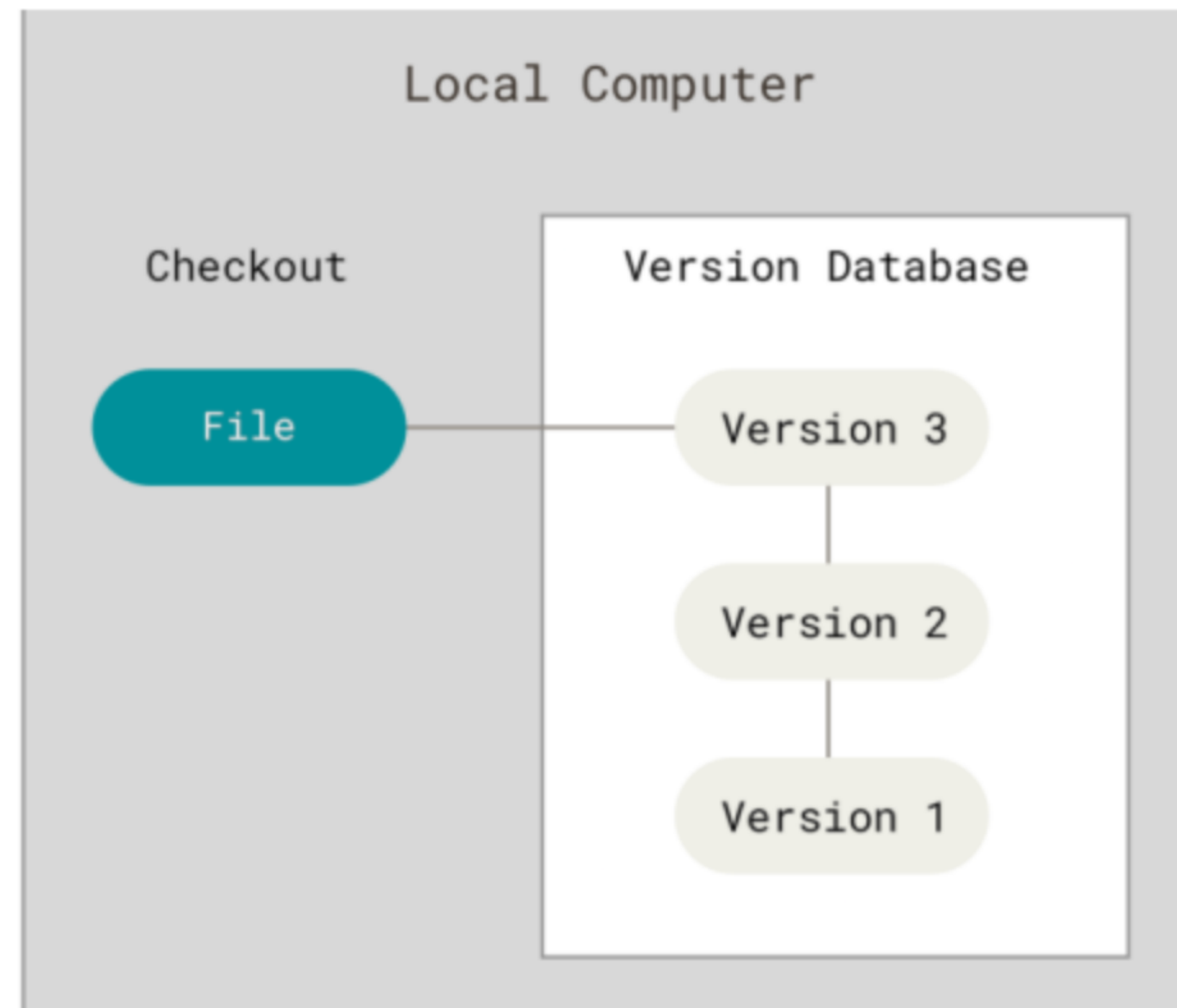
Why Version Control? The Problem

- ▶ Files named:
 - ▶ `final.c`
 - ▶ `final_v2.c`
 - ▶ `final_really_final.c`
- ▶ No clear history
- ▶ Difficult collaboration

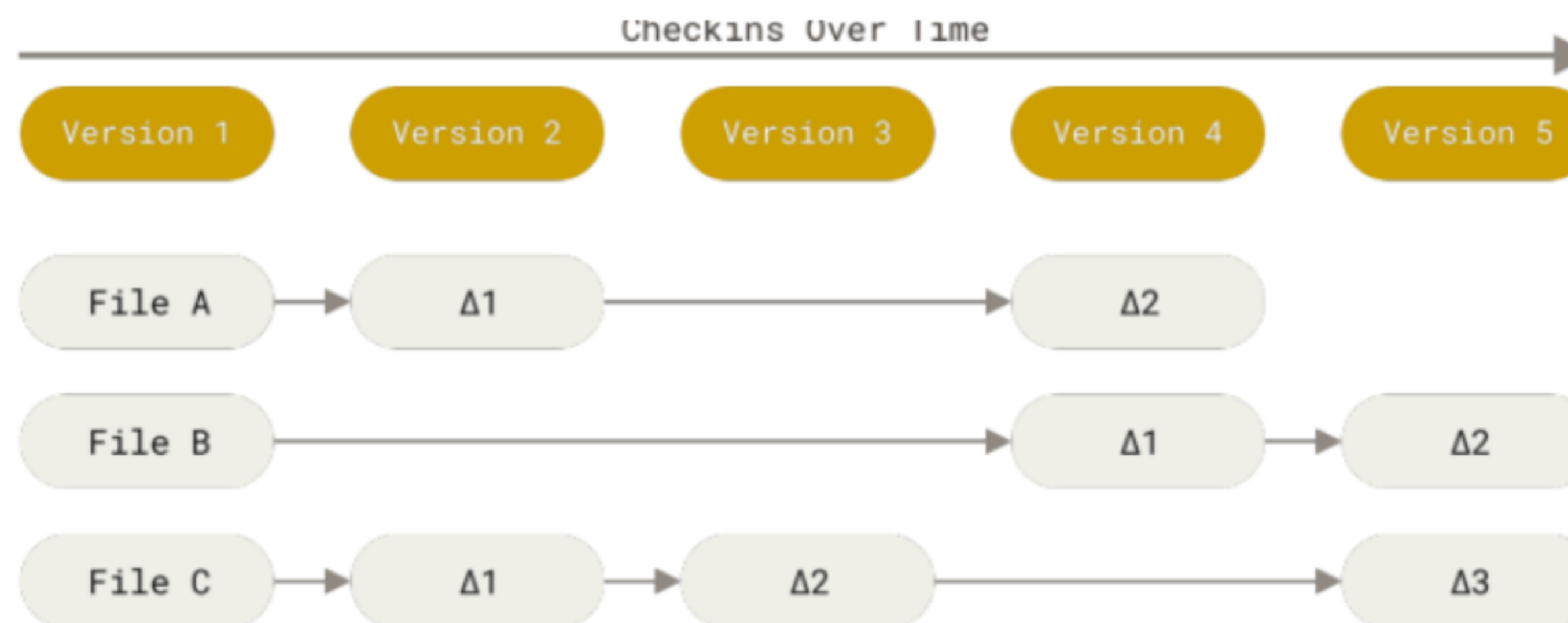
Why Version Control? The Problem

- ▶ Files named:
 - ▶ `final.c`
 - ▶ `final_v2.c`
 - ▶ `final_really_final.c`
- ▶ No clear history
- ▶ Difficult collaboration
- ▶ Easy to lose work

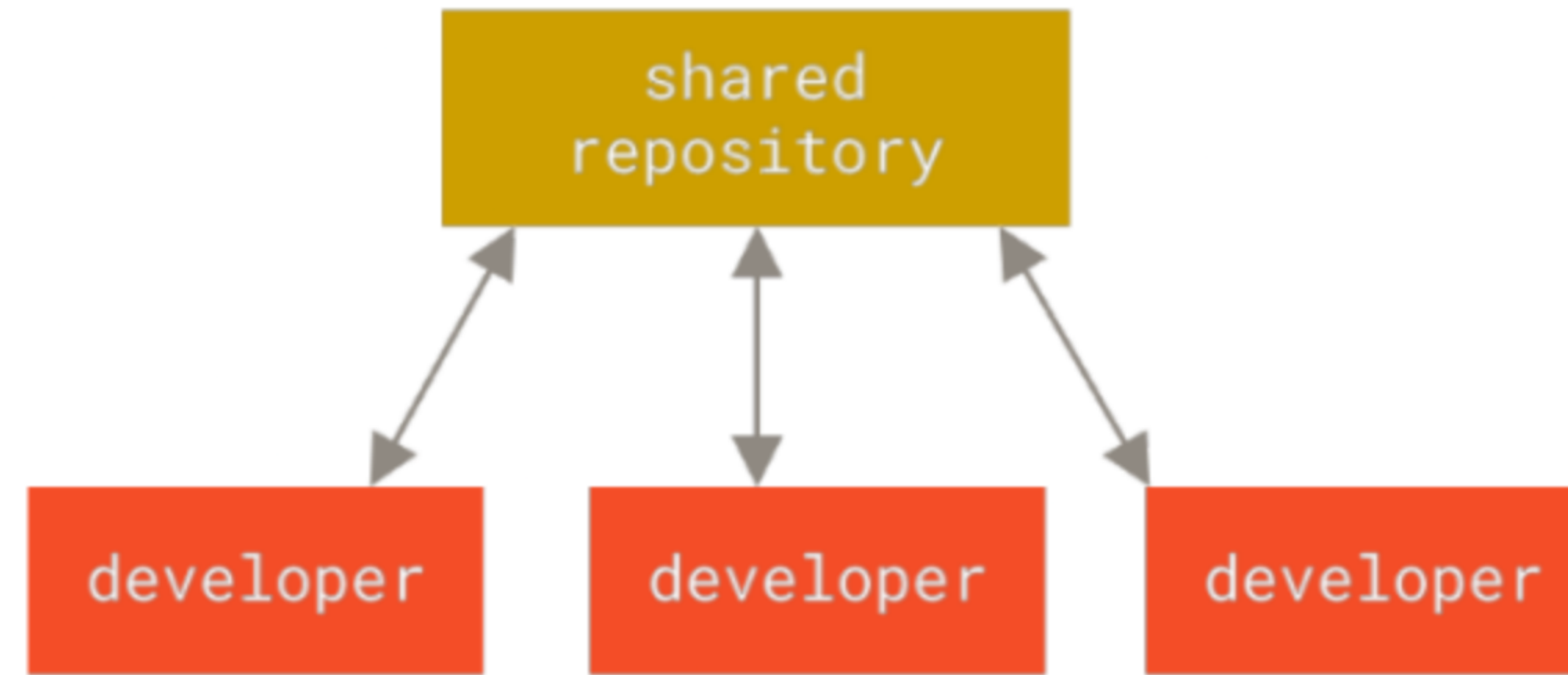
Local Version Control System



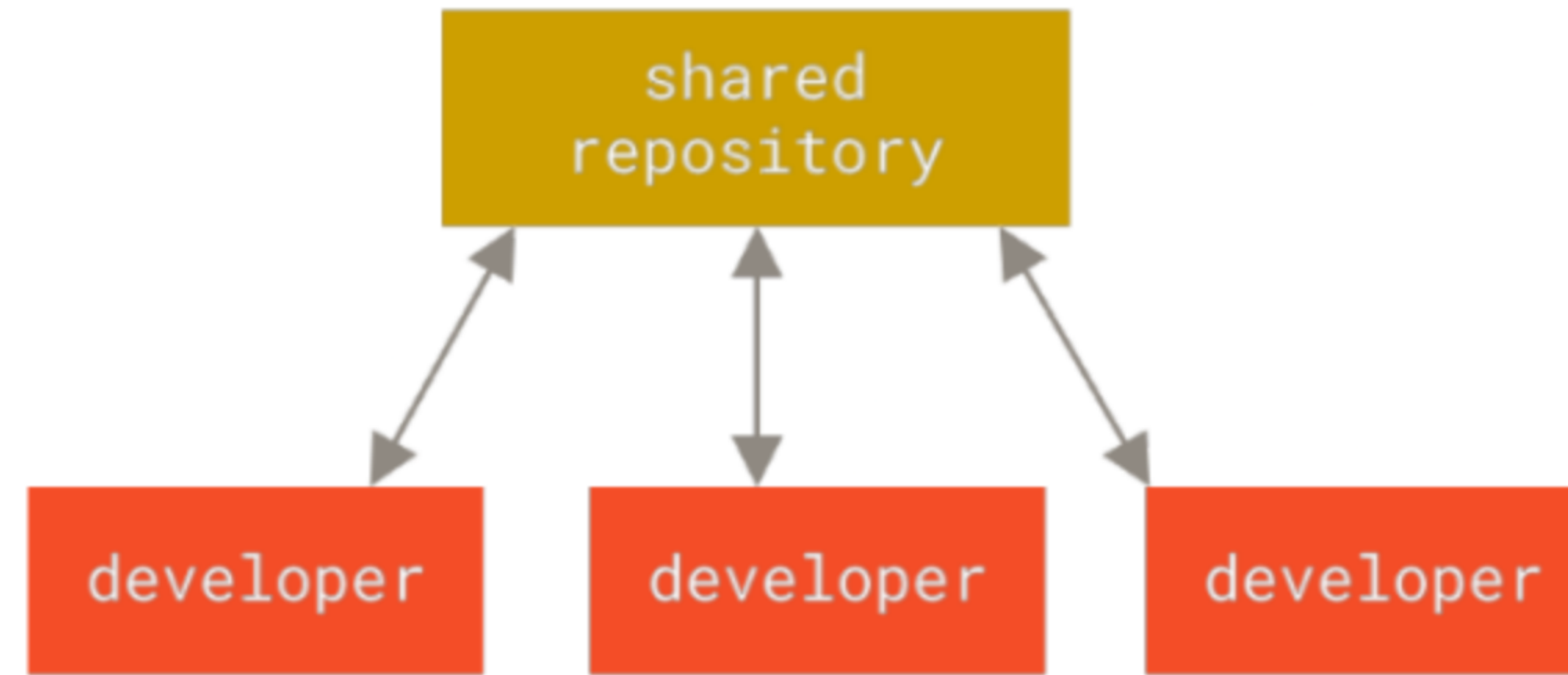
RCS - Idea: Keep patch sets that can be used to recreate a file



Centralized Version Control (SVN)

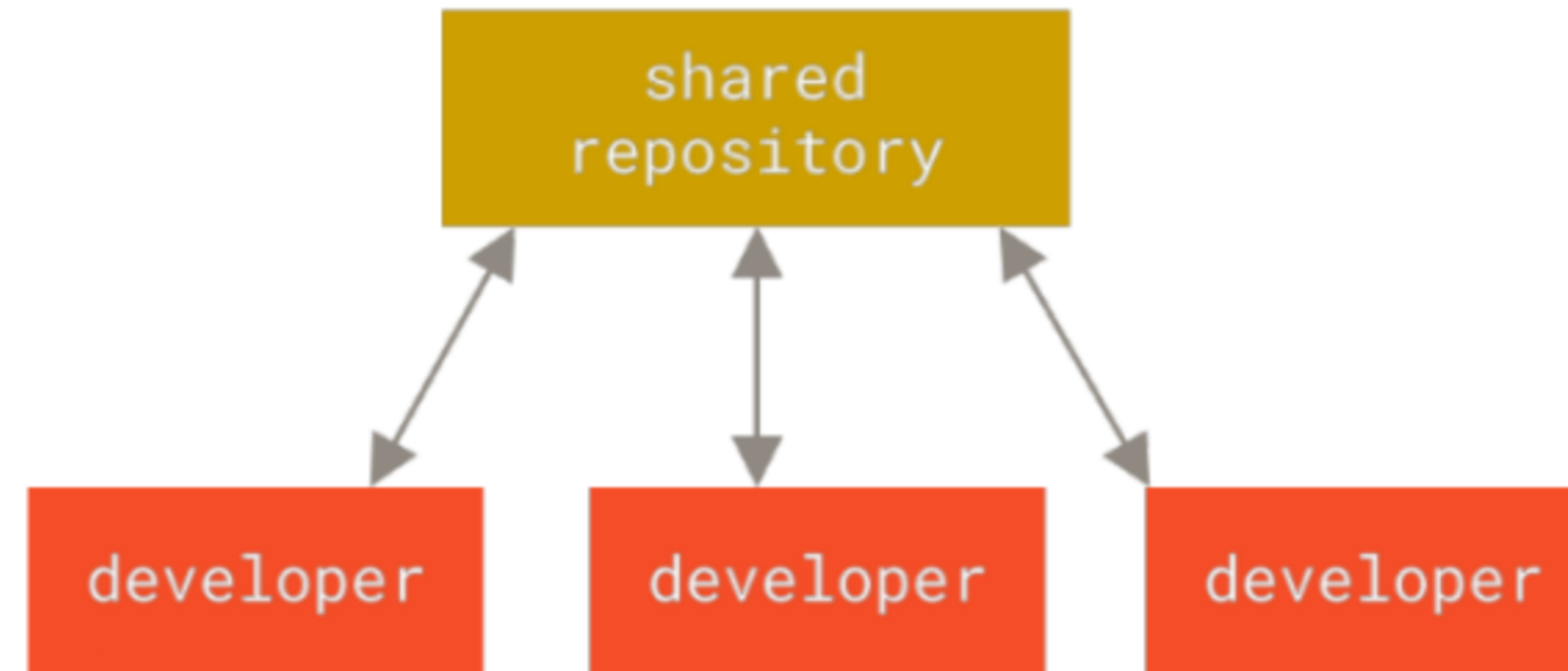


Centralized Version Control (SVN)



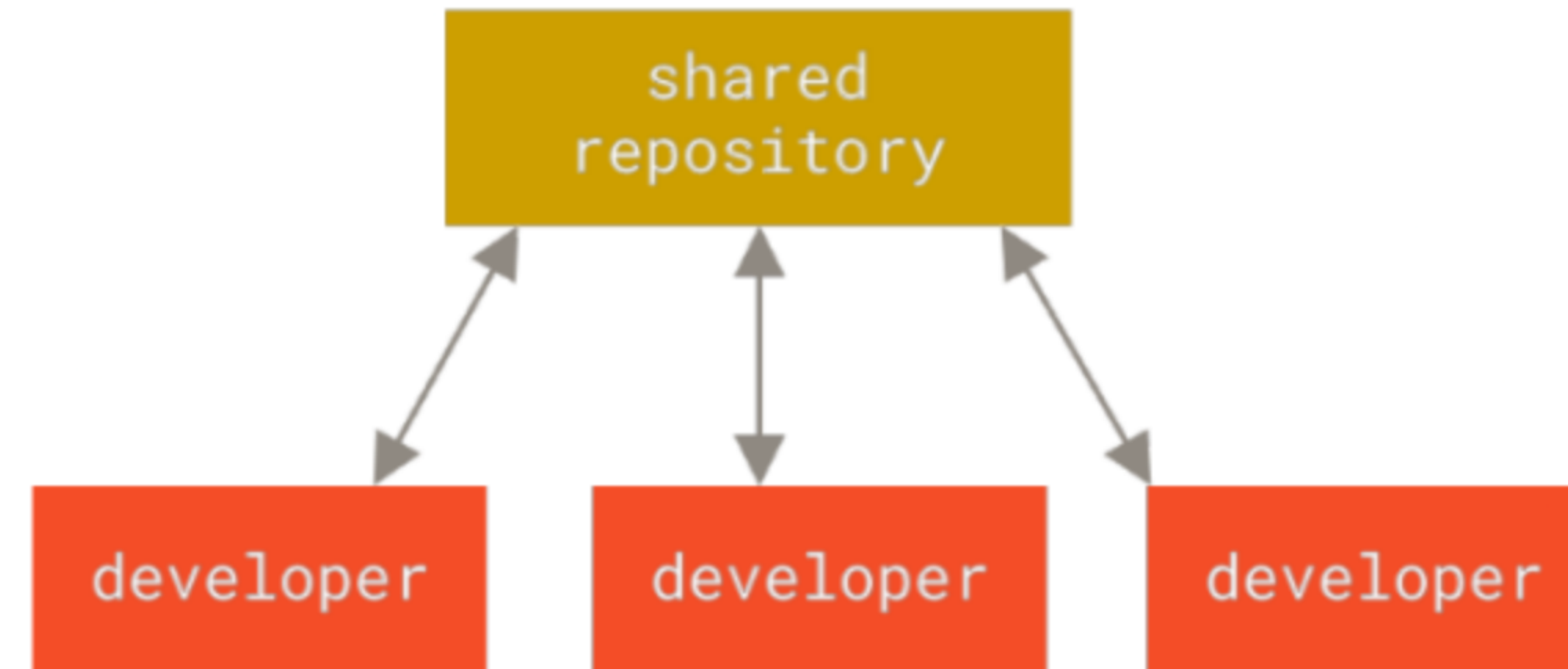
- ▶ Single central server

Centralized Version Control (SVN)



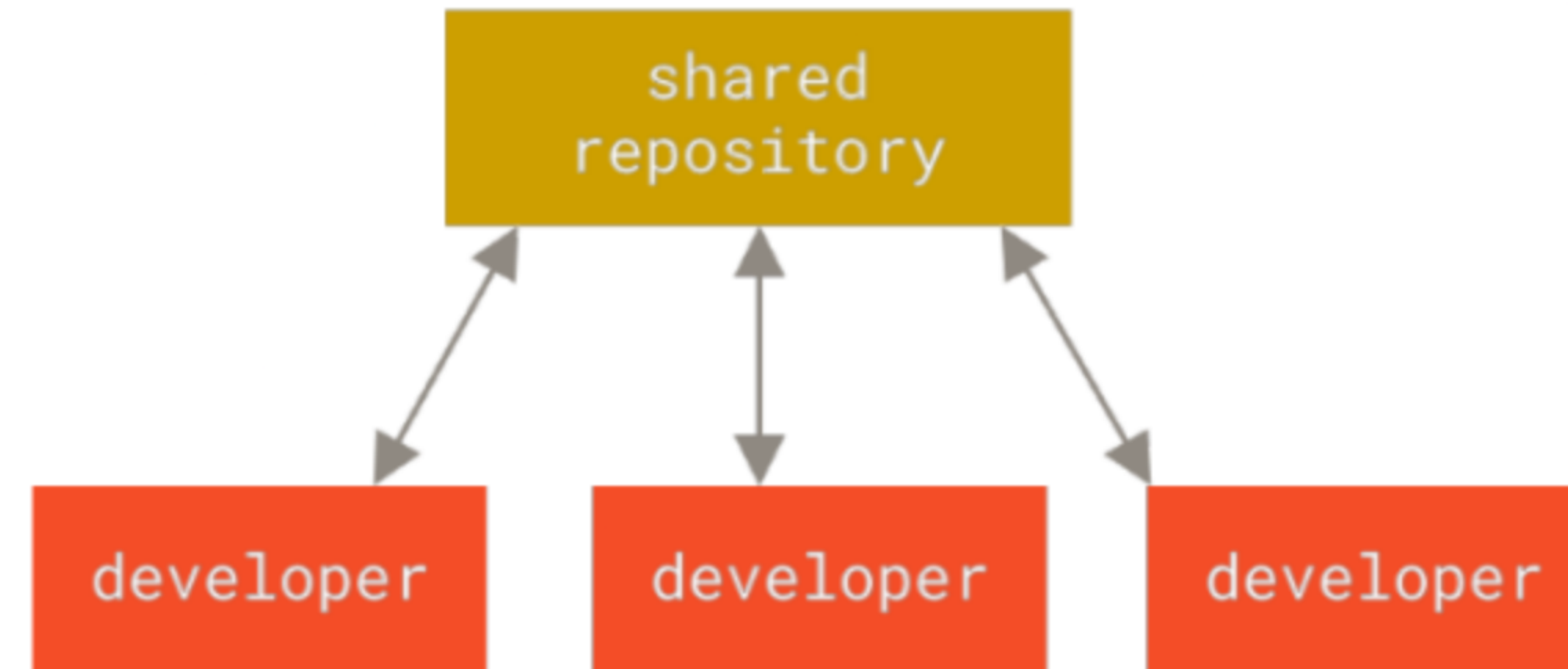
- ▶ Single central server
- ▶ Clients checkout working copies

Centralized Version Control (SVN)



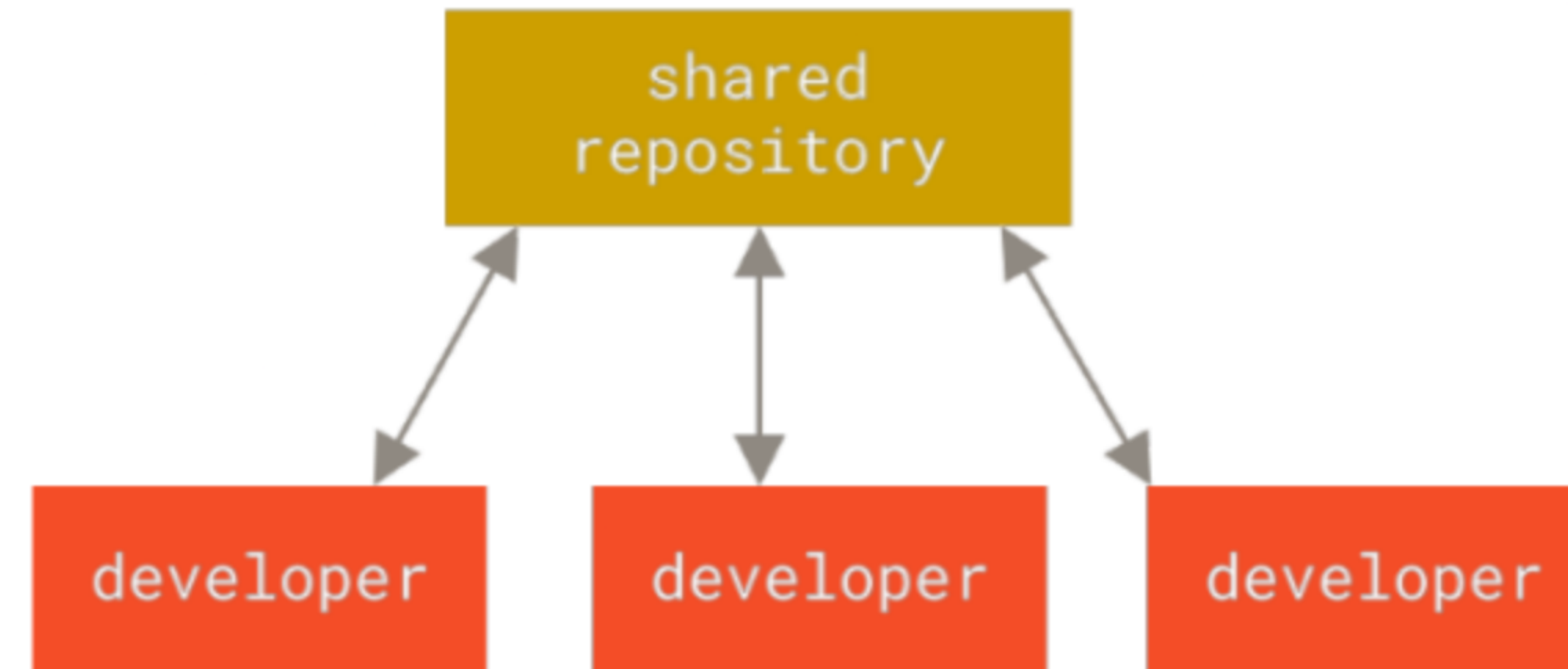
- ▶ Single central server
- ▶ Clients checkout working copies
- ▶ Commit requires network access

Centralized Version Control (SVN)



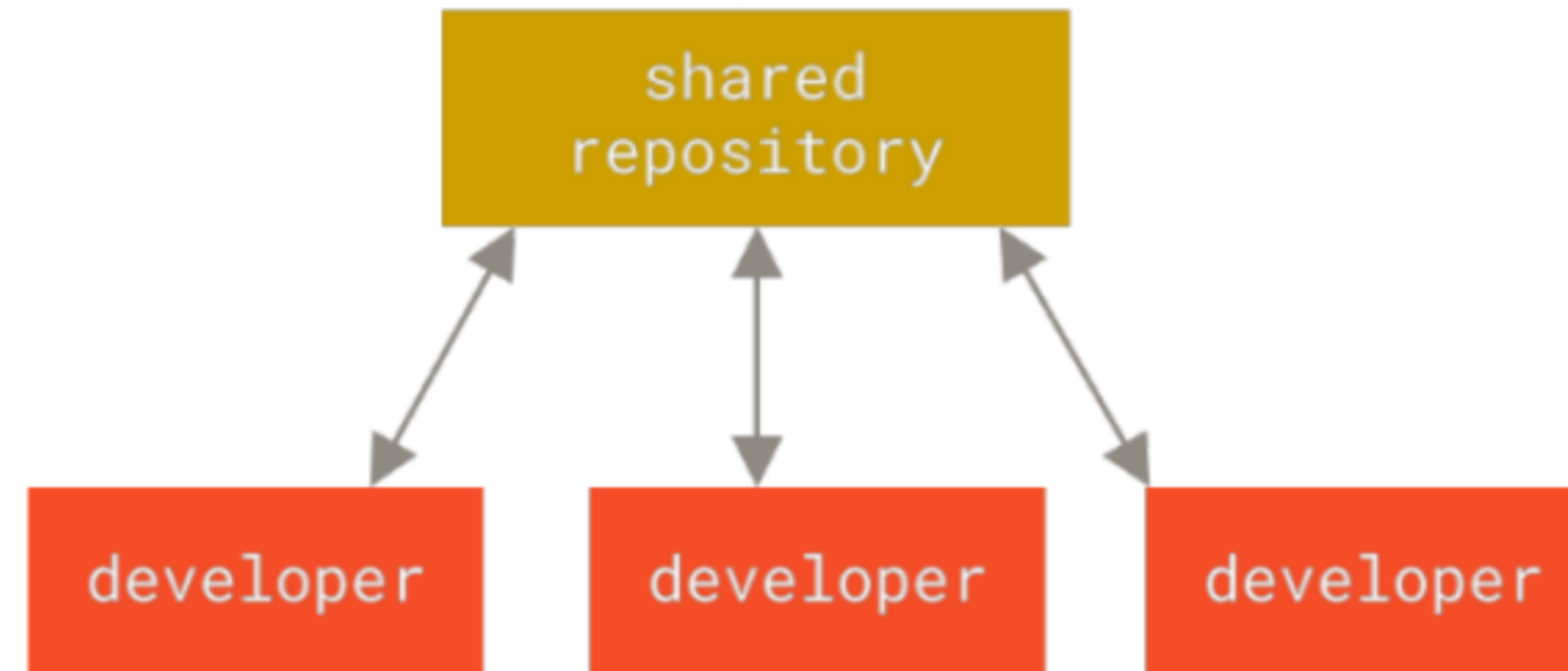
- ▶ Single central server
- ▶ Clients checkout working copies
- ▶ Commit requires network access
- ▶ Branching is expensive

Centralized Version Control (SVN)



- ▶ Single central server
- ▶ Clients checkout working copies
- ▶ Commit requires network access
- ▶ Branching is expensive

Centralized Version Control (SVN)

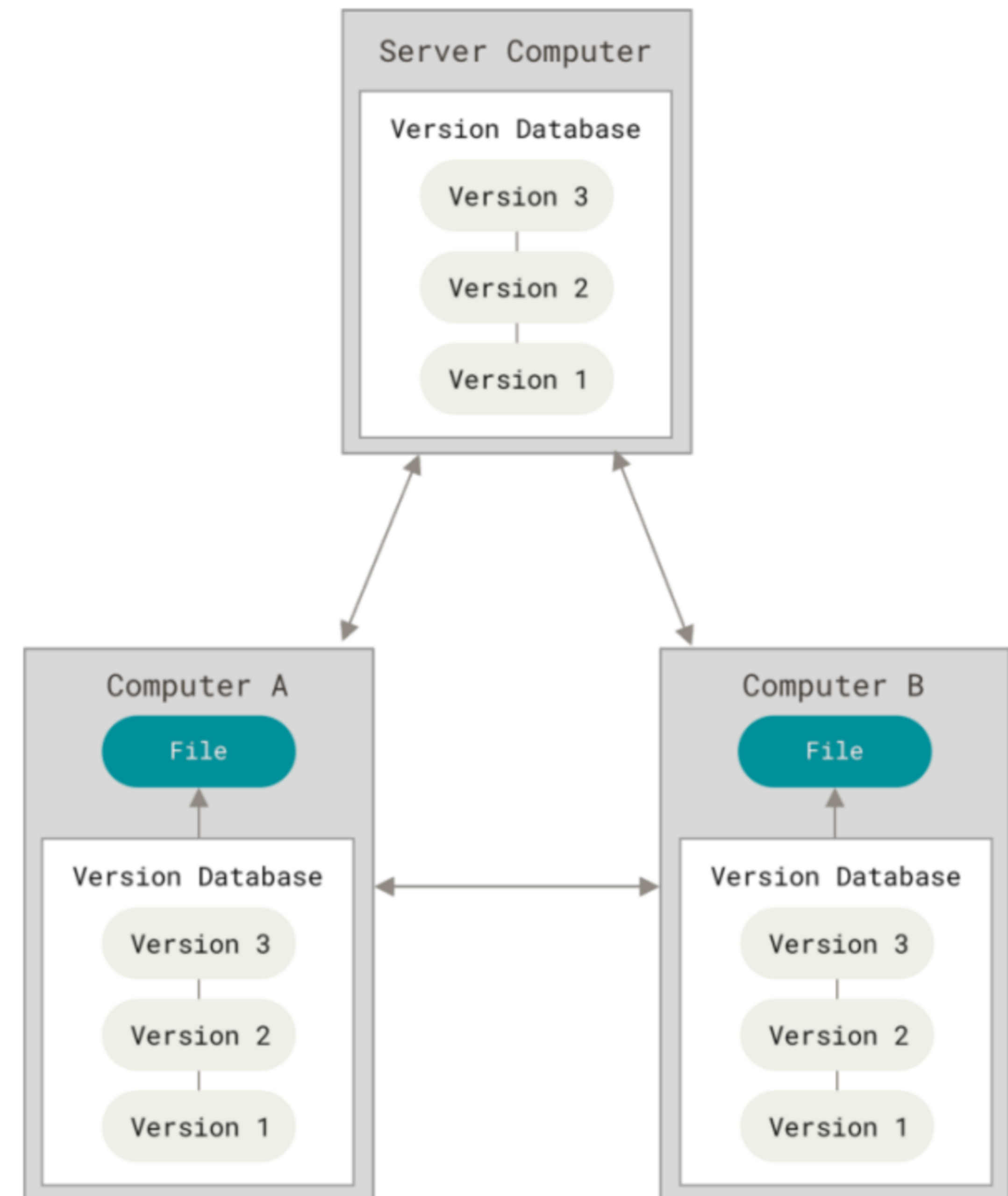


- ▶ Single central server
- ▶ Clients checkout working copies
- ▶ Commit requires network access
- ▶ Branching is expensive

Key Limitation: Single point of failure

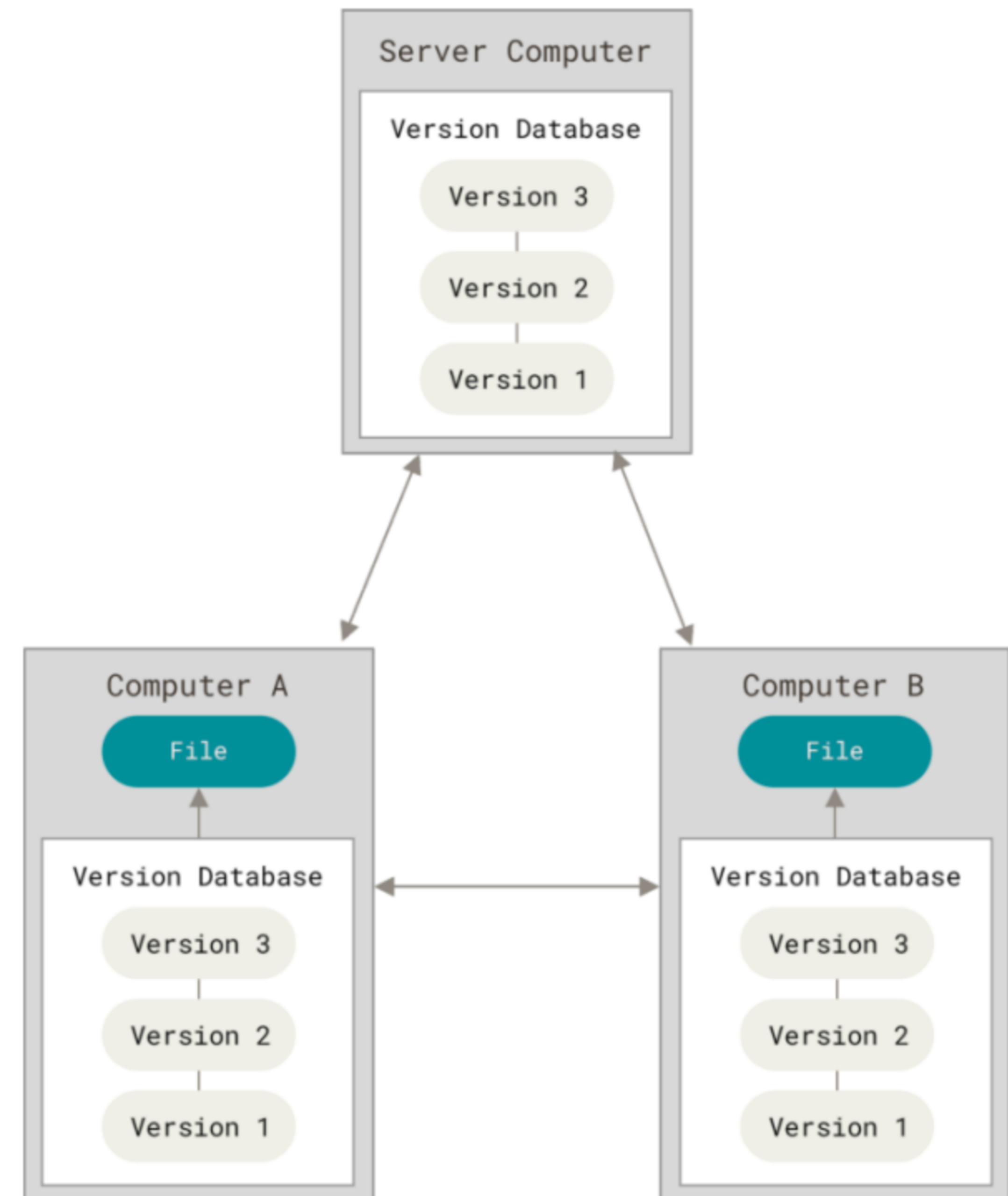
Why Git?

► Distributed version control



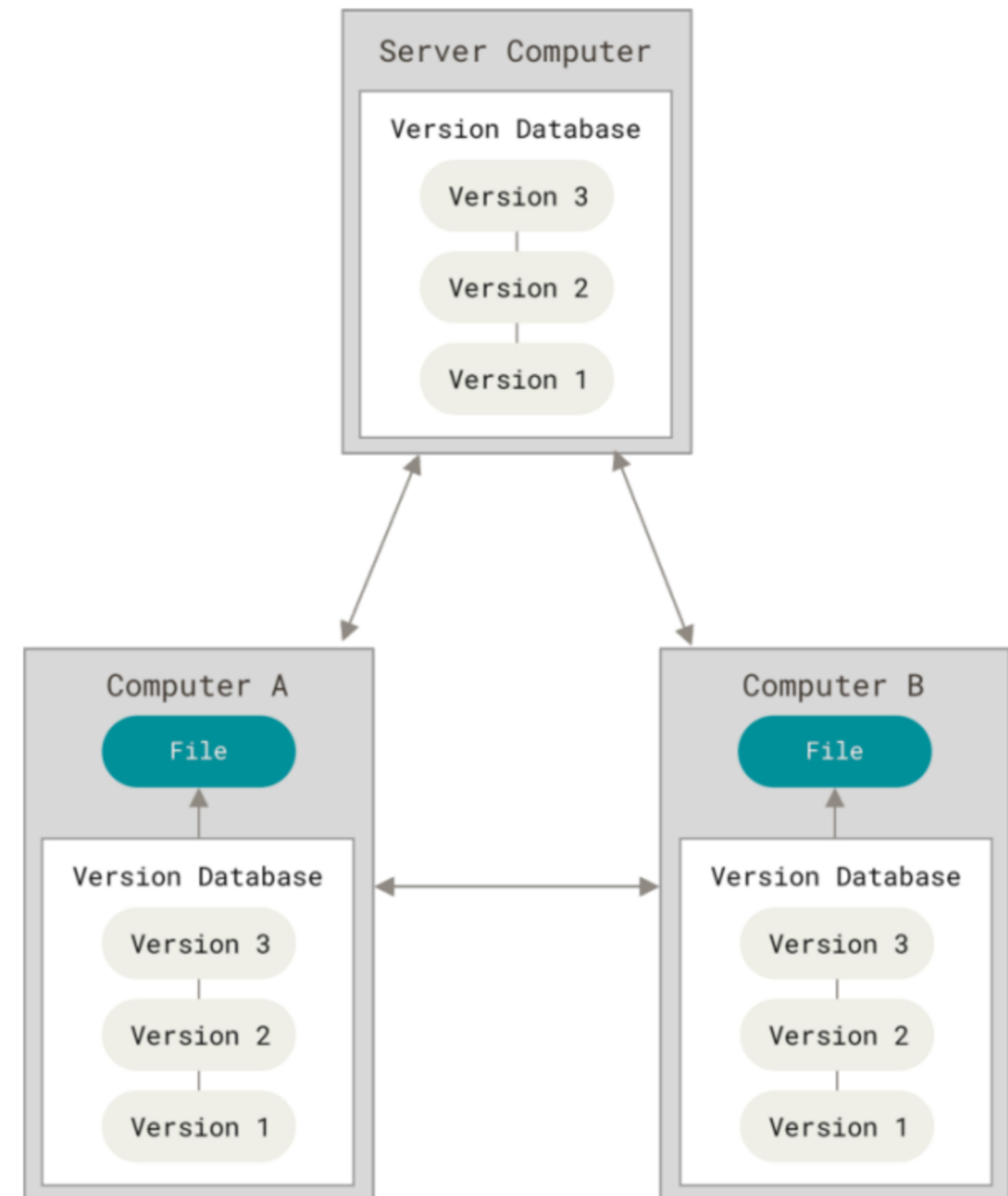
Why Git?

- ▶ Distributed version control
- ▶ Every clone has full history



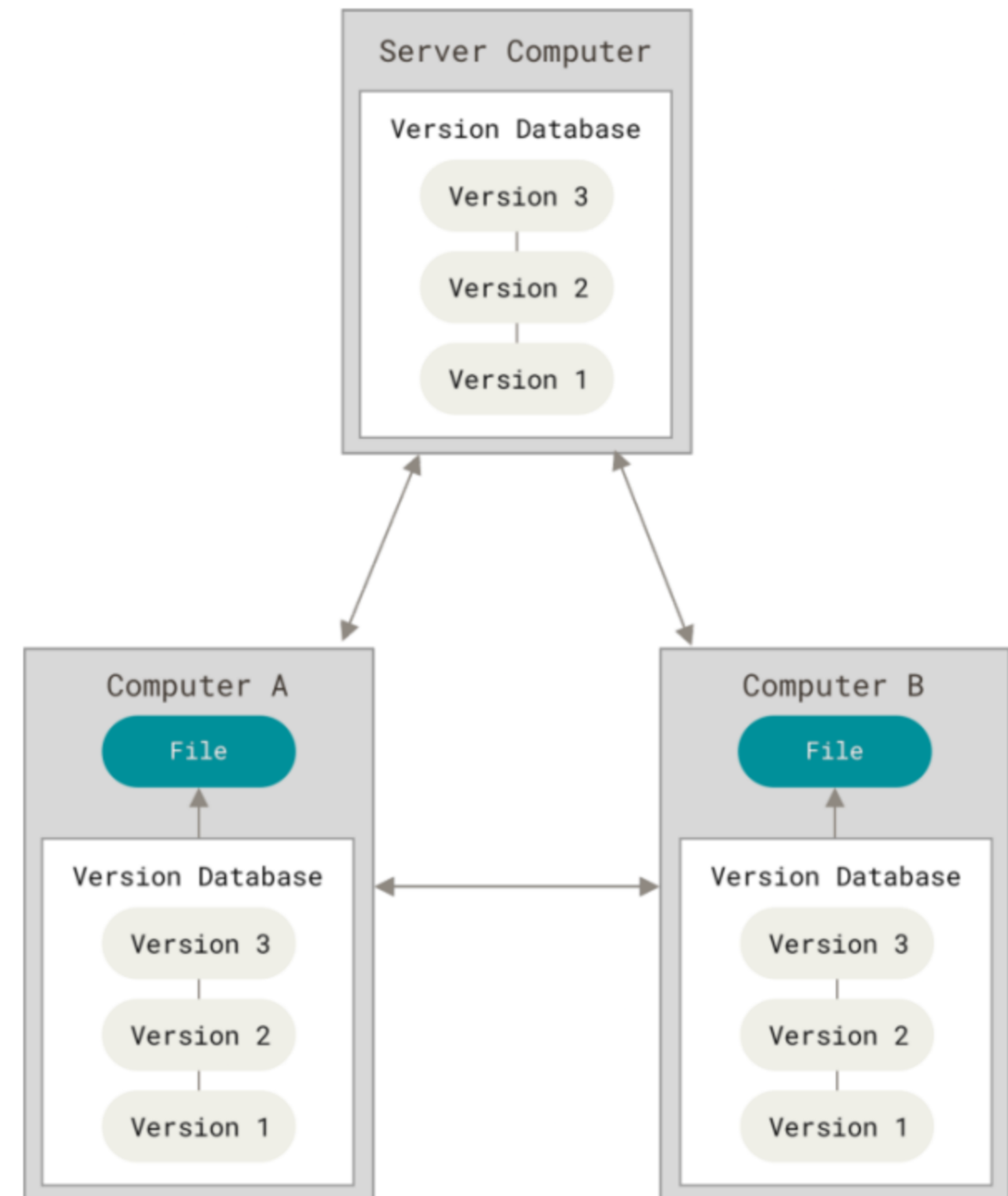
Why Git?

- ▶ Distributed version control
- ▶ Every clone has full history
- ▶ Fast, local commits



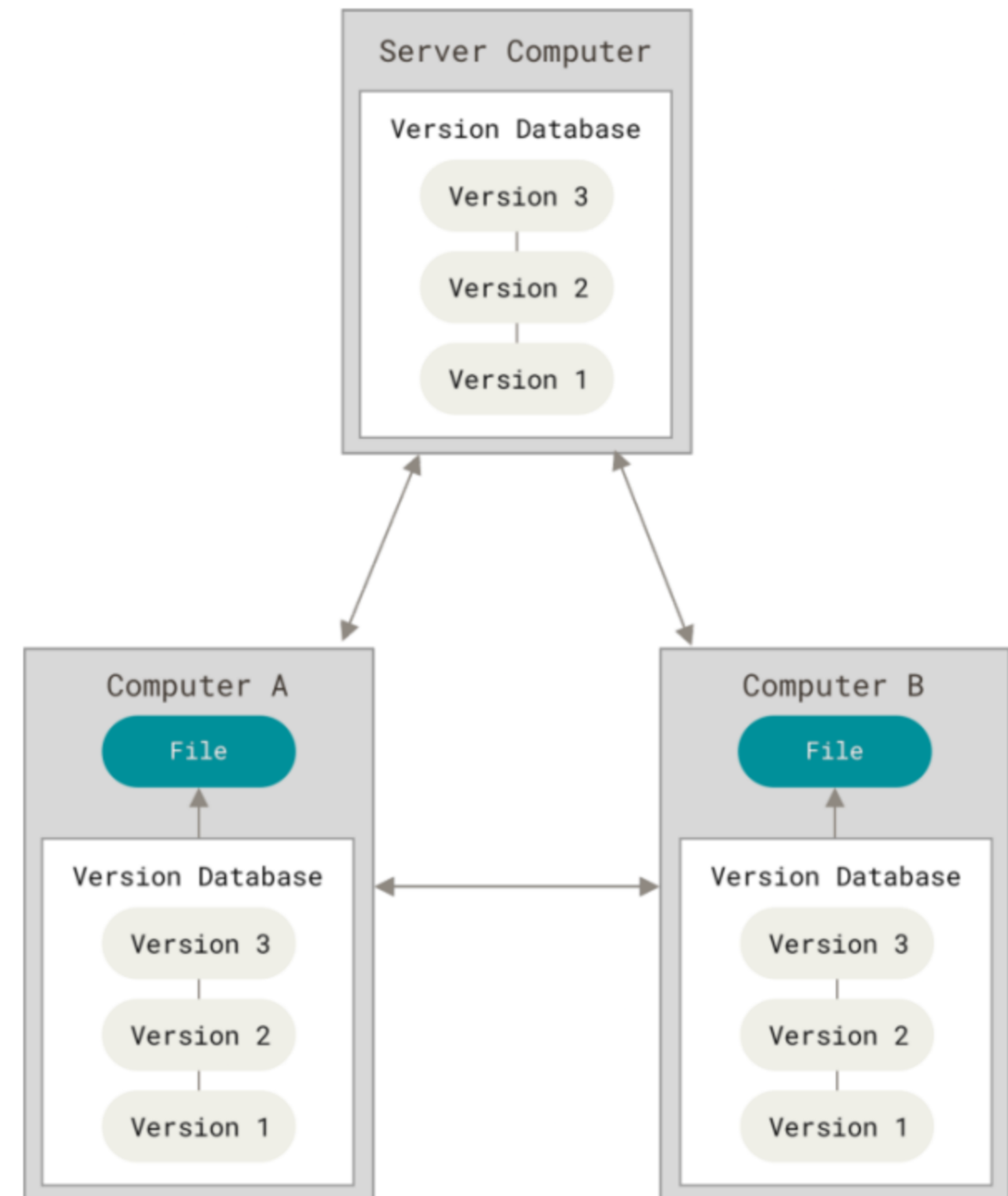
Why Git?

- ▶ Distributed version control
- ▶ Every clone has full history
- ▶ Fast, local commits
- ▶ Cheap branching and merging



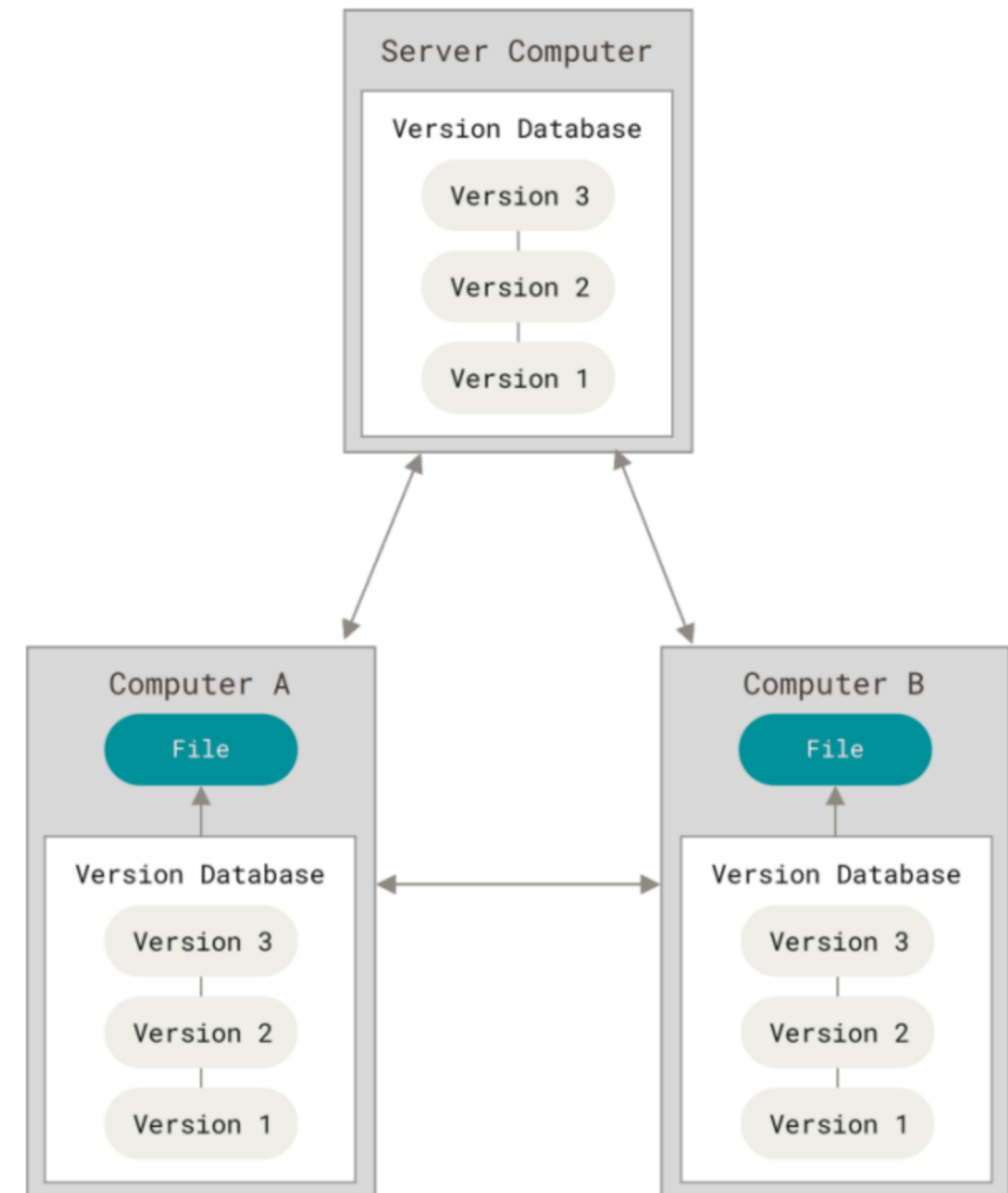
Why Git?

- ▶ Distributed version control
- ▶ Every clone has full history
- ▶ Fast, local commits
- ▶ Cheap branching and merging



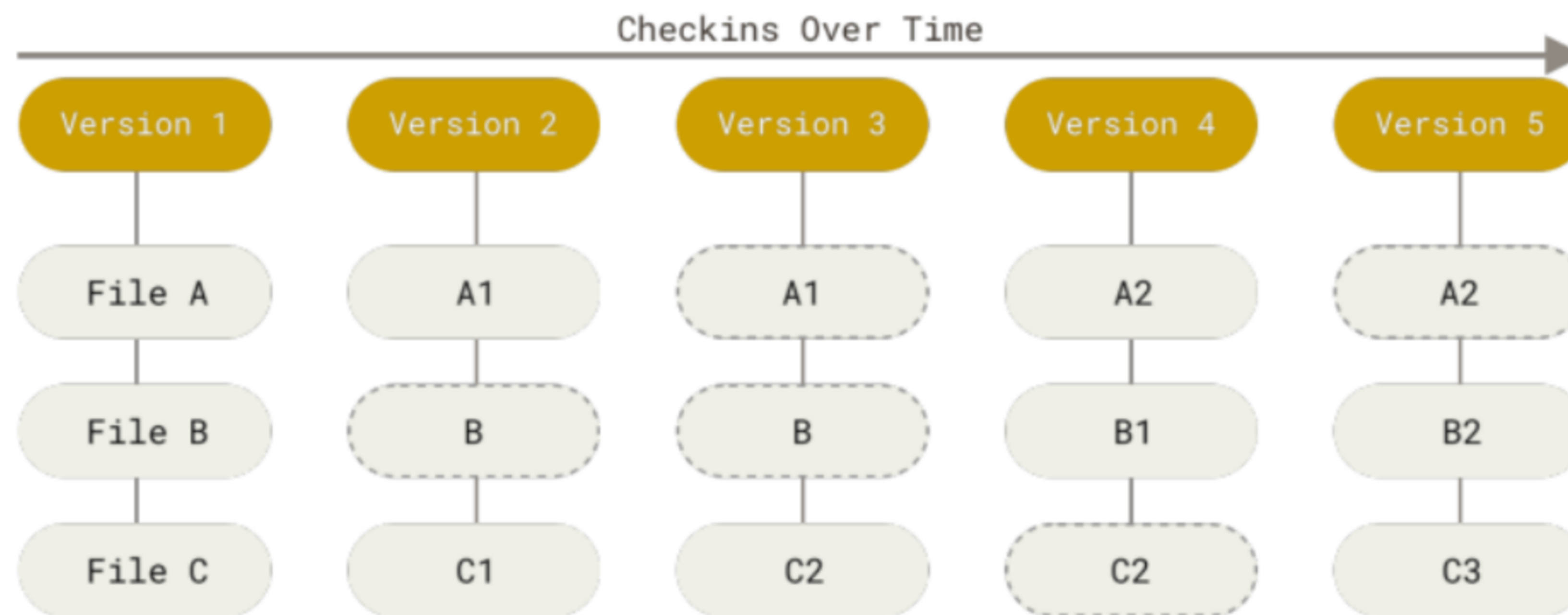
Why Git?

- ▶ Distributed version control
- ▶ Every clone has full history
- ▶ Fast, local commits
- ▶ Cheap branching and merging



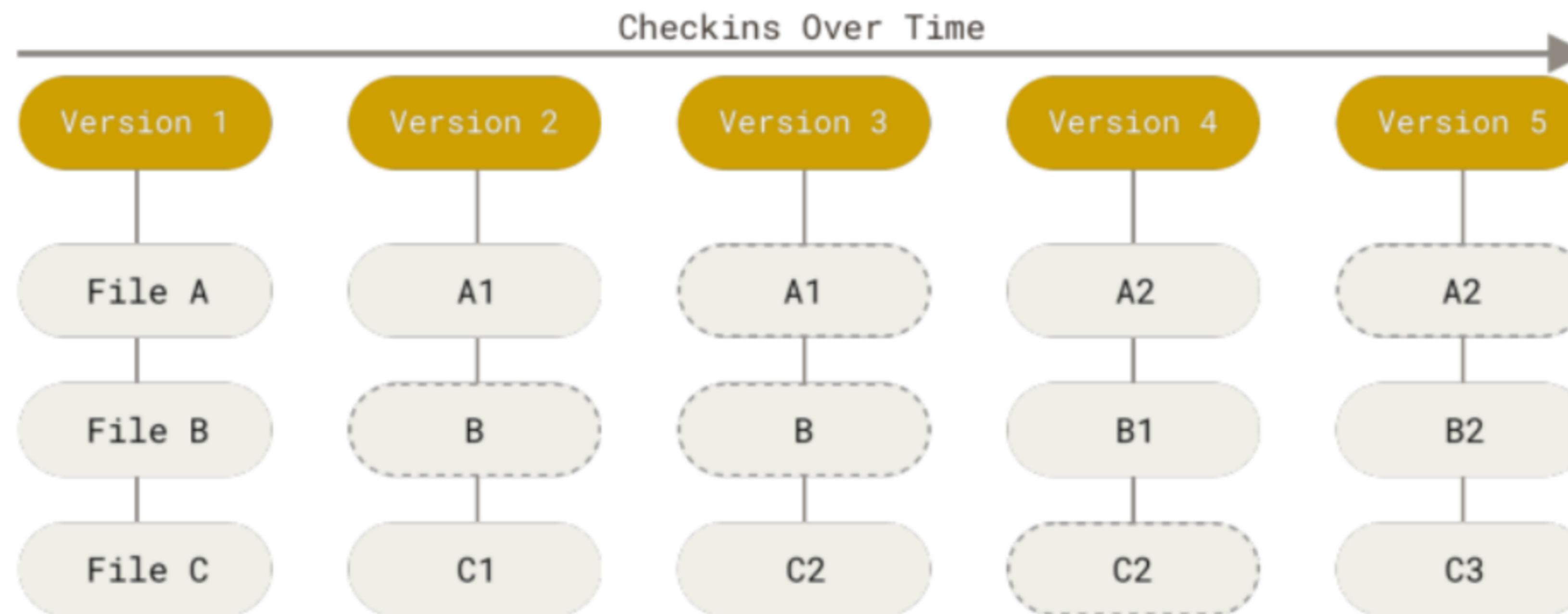
Key Idea: Collaboration without central dependency

Git's Mental Model



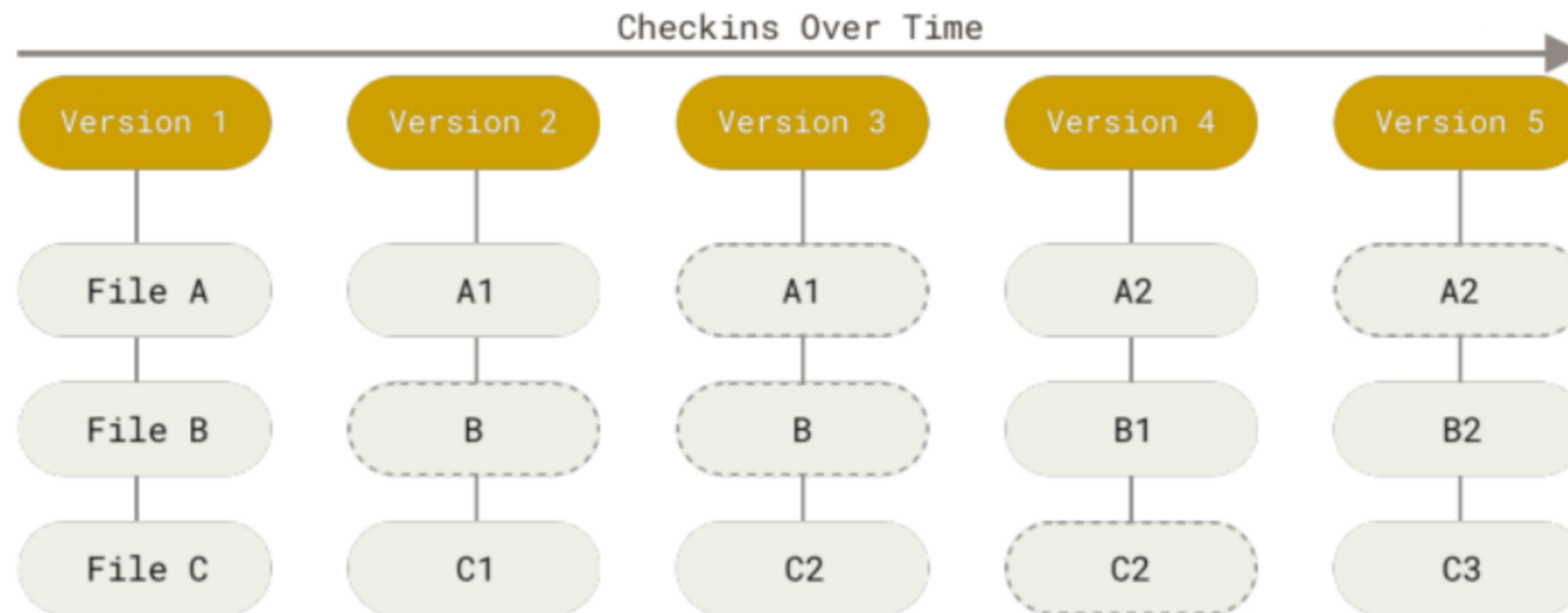
- ▶ Git stores **snapshots**, not diffs

Git's Mental Model



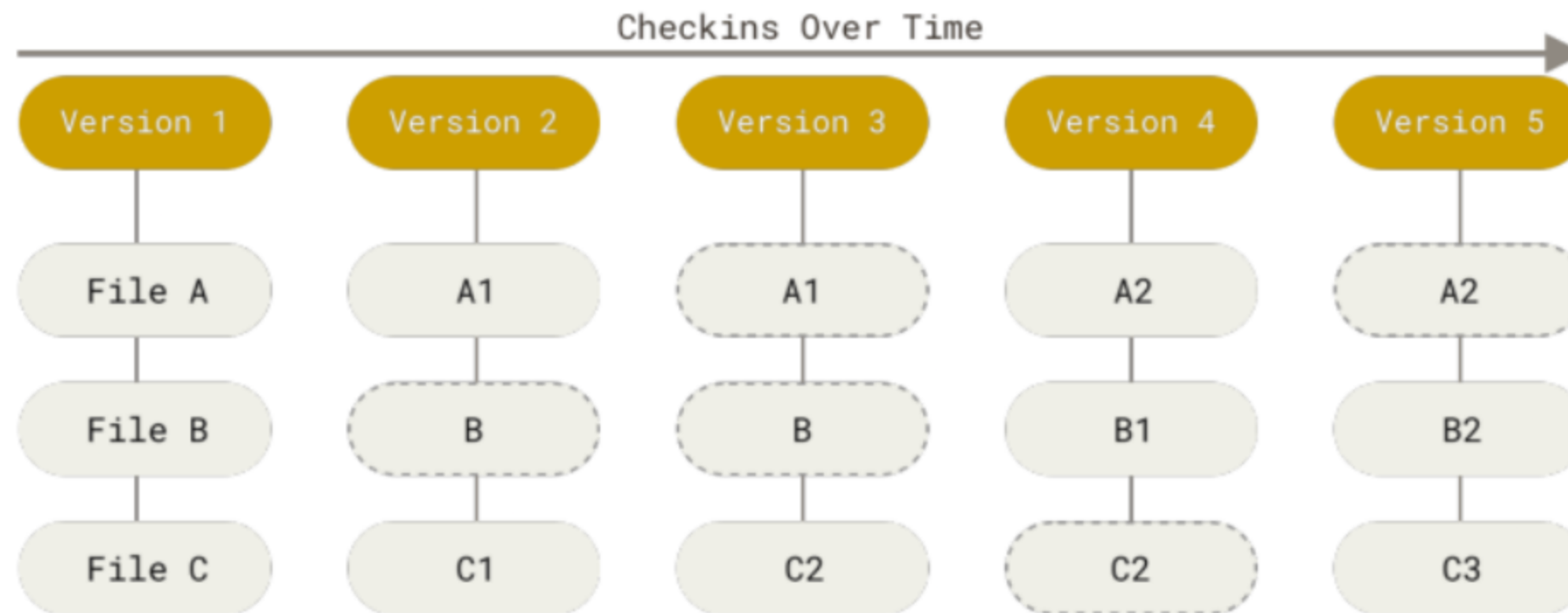
- ▶ Git stores **snapshots**, not diffs
- ▶ Each commit points to:

Git's Mental Model



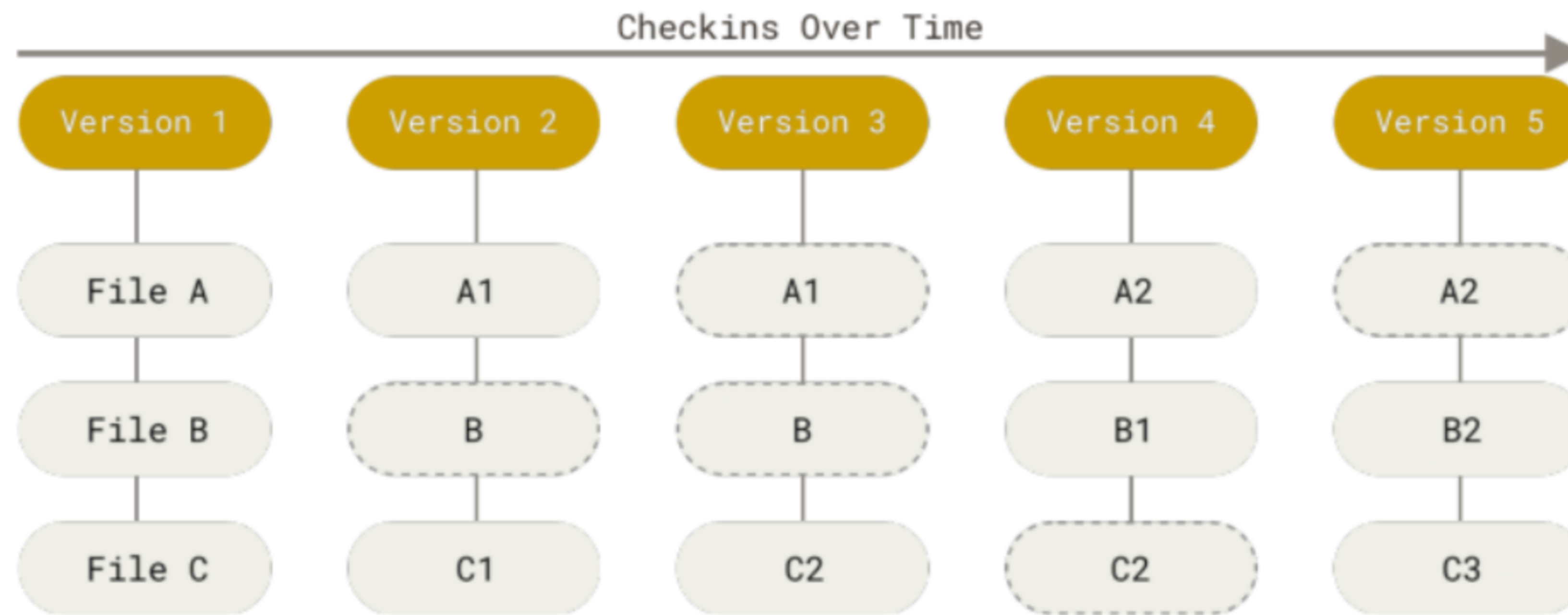
- ▶ Git stores **snapshots**, not diffs
- ▶ Each commit points to:
 - ▶ A snapshot of files

Git's Mental Model



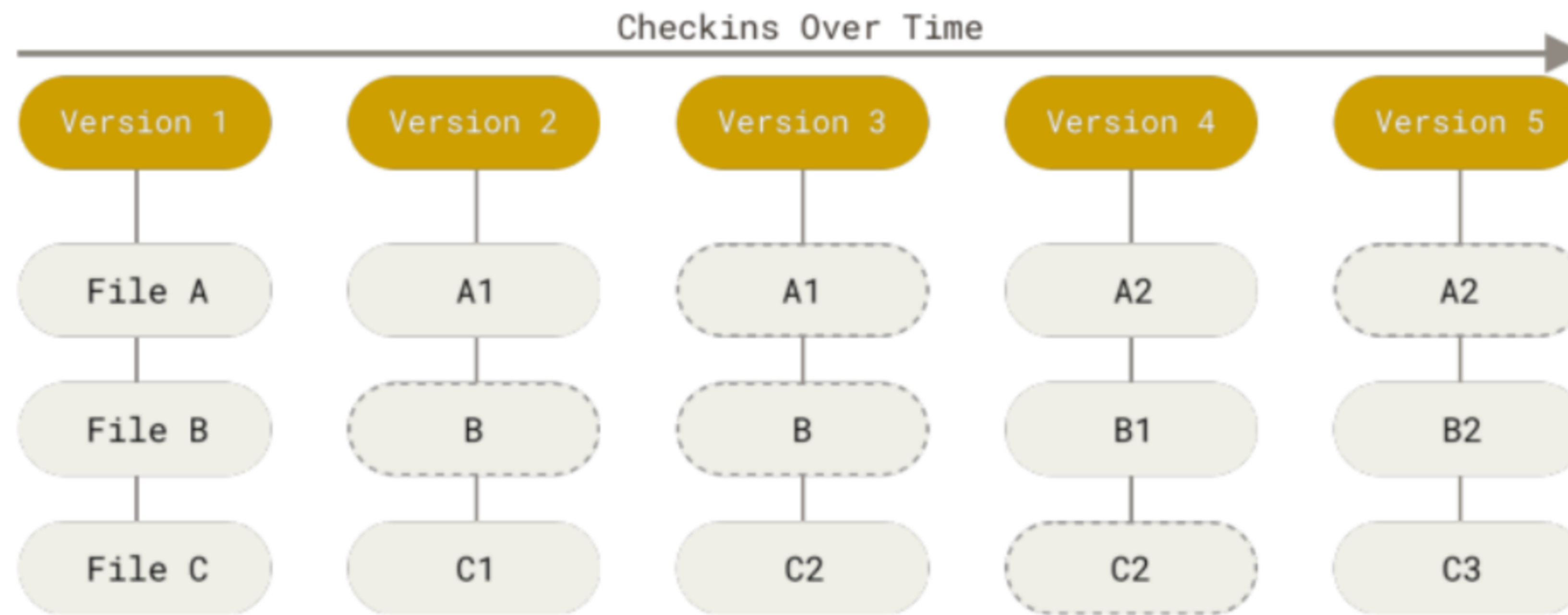
- ▶ Git stores **snapshots**, not diffs
- ▶ Each commit points to:
 - ▶ A snapshot of files
 - ▶ Parent commit(s)

Git's Mental Model



- ▶ Git stores **snapshots**, not diffs
- ▶ Each commit points to:
 - ▶ A snapshot of files
 - ▶ Parent commit(s)
- ▶ History forms a DAG (Directed Acyclic Graph)

Git's Mental Model



- ▶ Git stores **snapshots**, not diffs
- ▶ Each commit points to:
 - ▶ A snapshot of files
 - ▶ Parent commit(s)
- ▶ History forms a DAG (Directed Acyclic Graph)
- ▶ Nearly all operations are local

Git Configuration and Initializing a Repository

Commands

```
git config --global user.name "Kalpesh Kapoor"  
git config --global user.email kalpesh.kapoor@iiserpune.ac.in  
mkdir dspractice  
cd dspractice  
git init
```

- ▶ git **global** configuration in home directory ~/.gitconfig

```
[user]
```

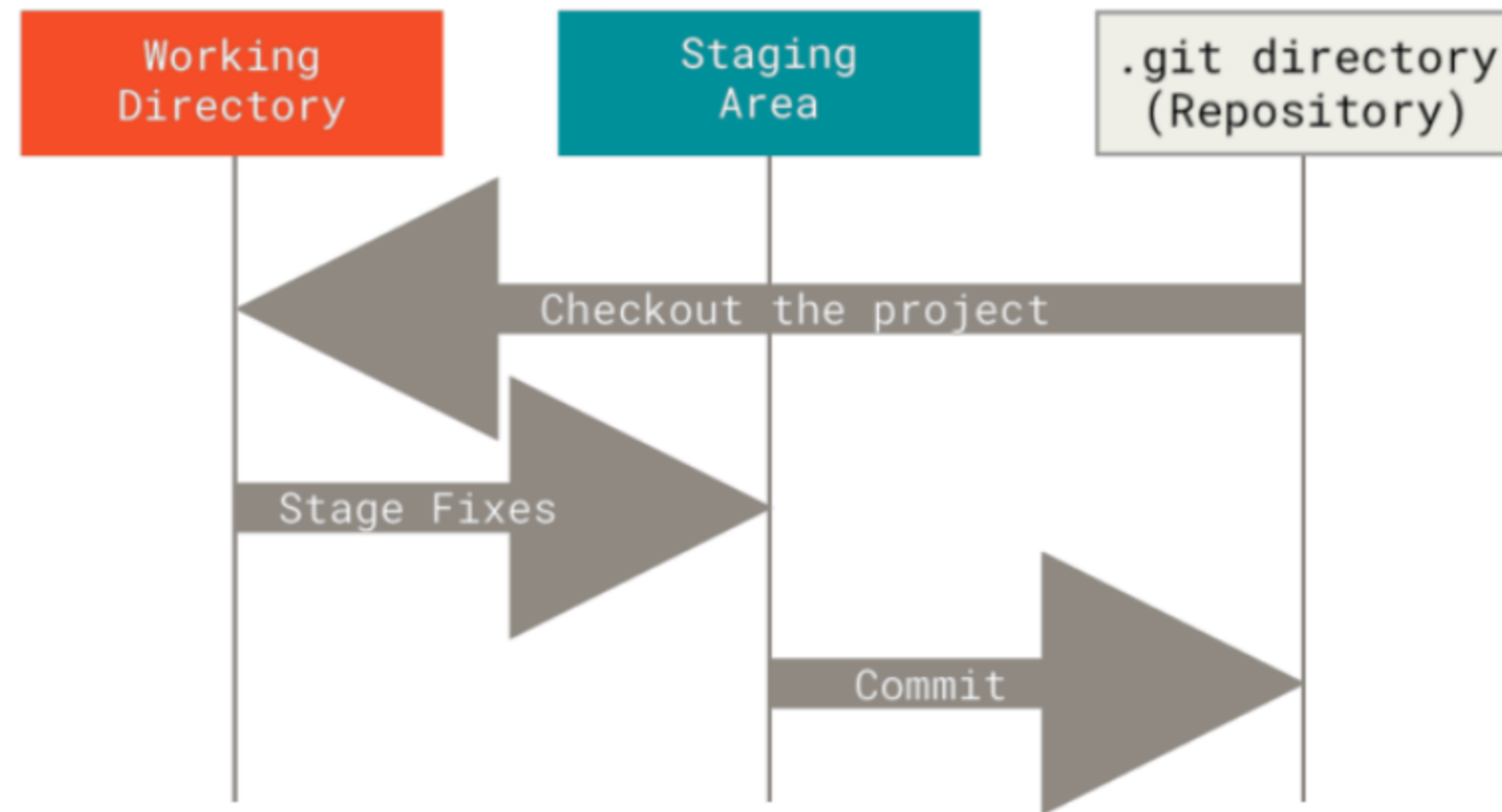
```
email = kalpesh.kapoor@iiserpune.ac.in
```

```
name  = Kalpesh
```

```
...
```

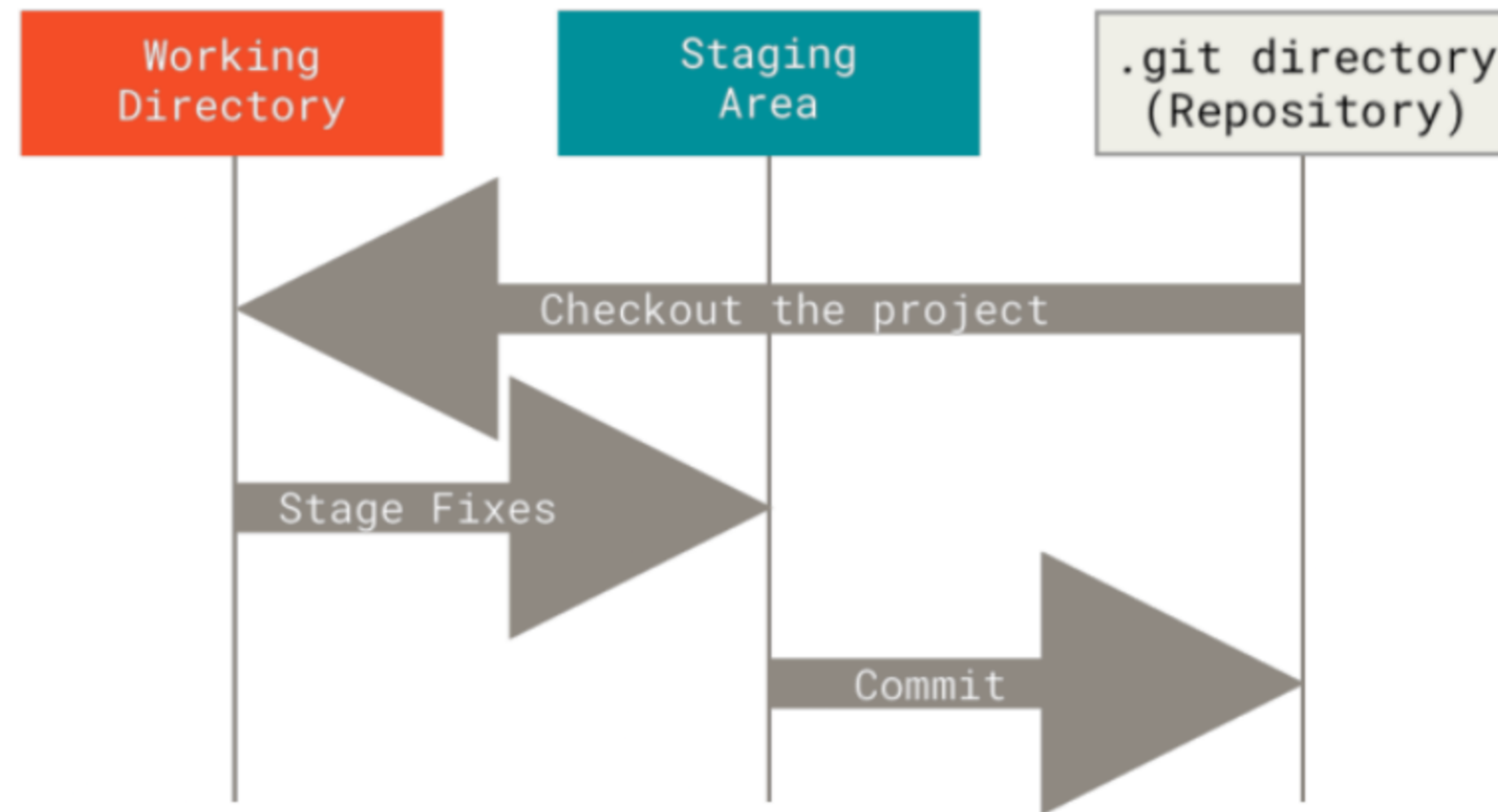
- ▶ Creates a .git directory
- ▶ Working directory vs Repository

The Three States of Files



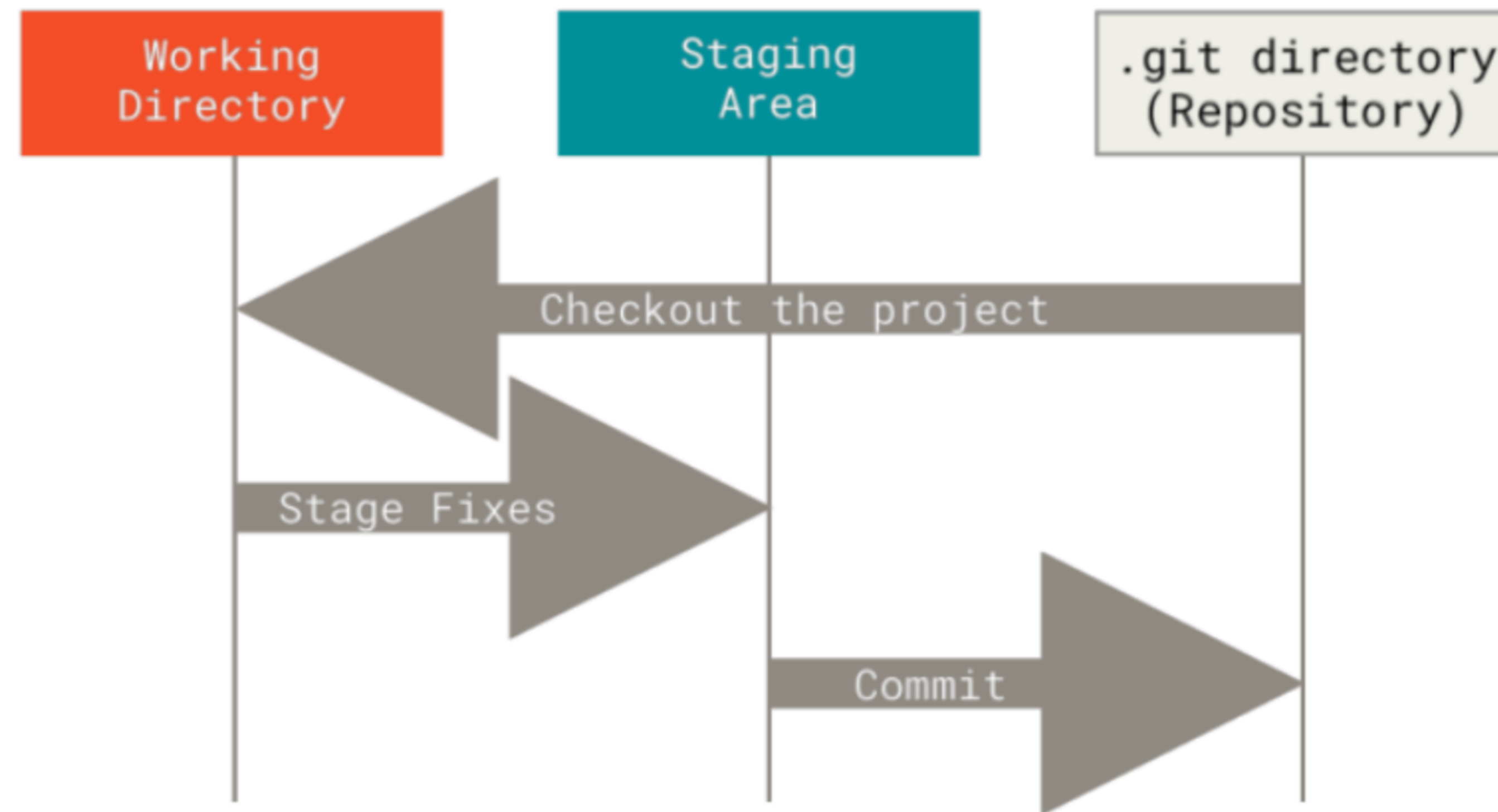
► Modified

The Three States of Files



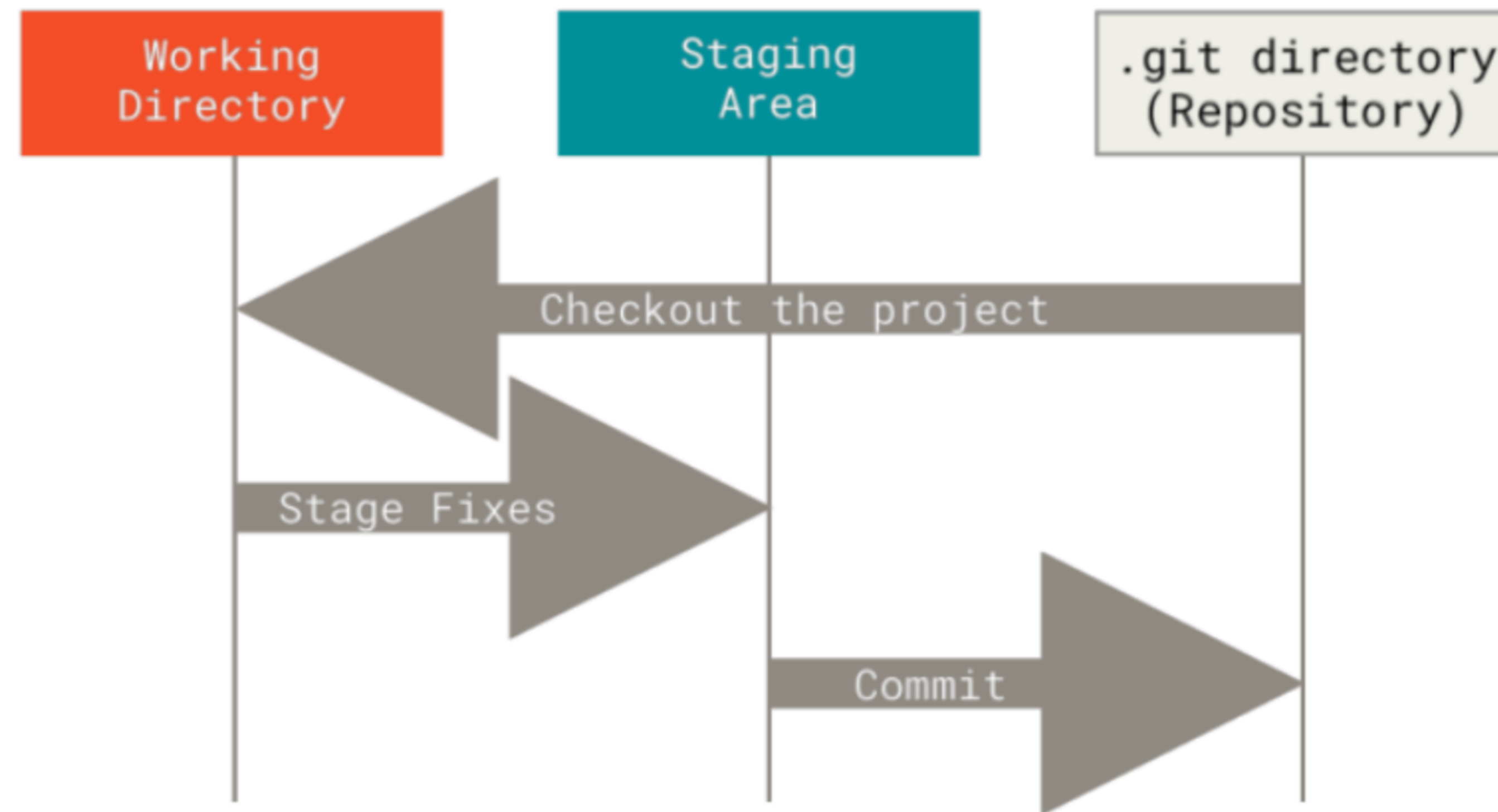
- ▶ Modified
- ▶ Staged

The Three States of Files



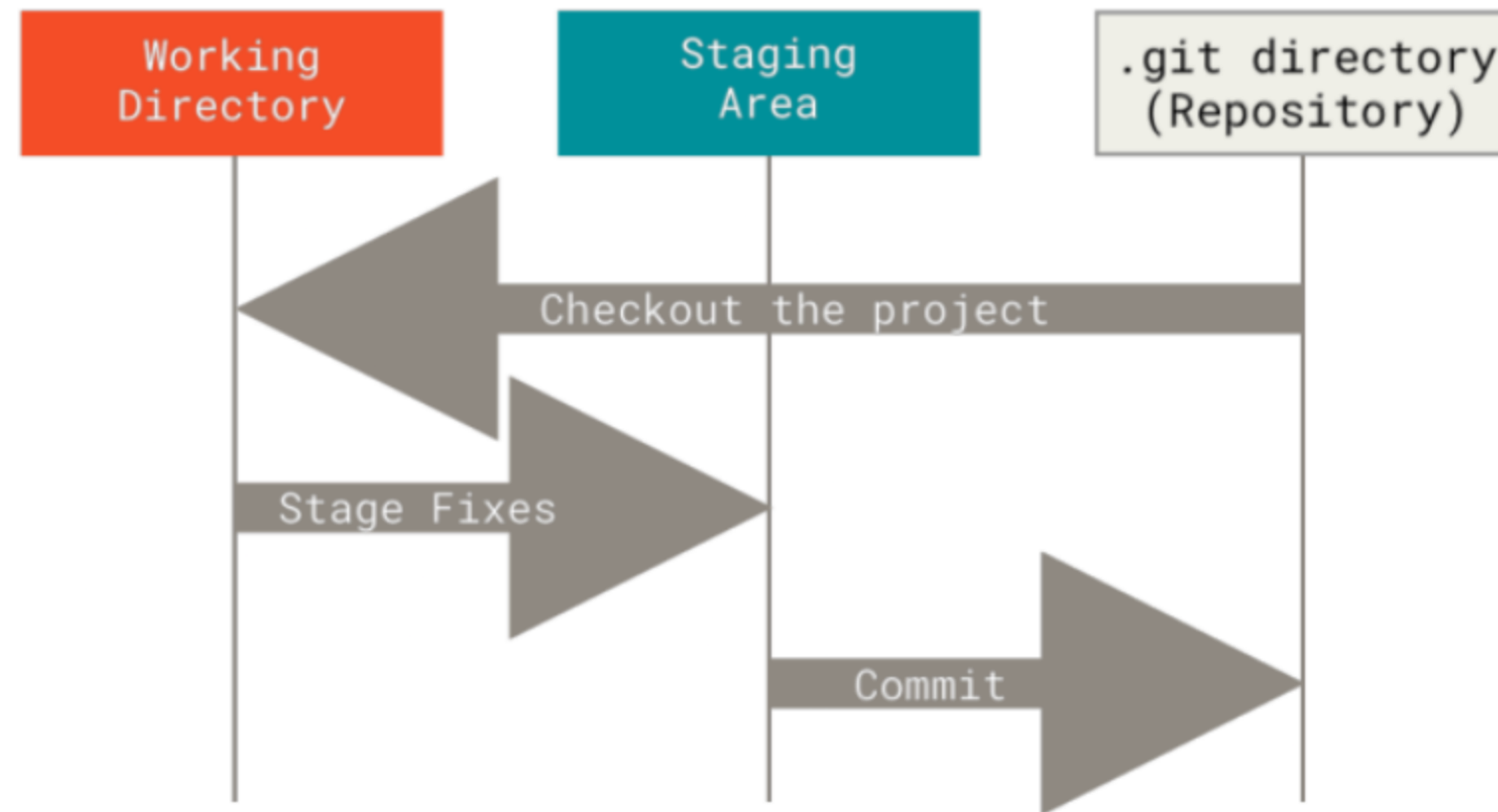
- ▶ Modified
- ▶ Staged
- ▶ Committed

The Three States of Files



- ▶ Modified
- ▶ Staged
- ▶ Committed

The Three States of Files



- ▶ Modified
- ▶ Staged
- ▶ Committed

Staging Area = deliberate commit preparation

First Commit

Commands

```
echo "Version 1" > notes.txt  
git status  
git add notes.txt  
git commit -m "Initial commit"
```

- ▶ Commit = snapshot
- ▶ Commit message explains intent
- ▶ Git maintains integrity using hashes

Modified but Unstaged

Commands

```
echo "Version 2" >> notes.txt  
git status  
git diff
```

After add: `git diff --staged`

Question:

Why does Git not automatically commit this change?

Partial Staging, Ignoring and Removing Files, Logs

Commands

```
git add -p
```

- ▶ Stage changes interactively
- ▶ Build logical commits

Ignore files: Specify patterns in `.gitignore`

Remove files: `git rm [--cached] file`

Moving files: `git mv old new`

Explore: `git log`, `git log -p`

Undoing

Commands

```
git commit -m 'Initial commit'
```

```
git add forgotten_file
```

```
git commit --amend
```

```
git restore --staged file      # for unstaging
```

```
git restore file      # to discard changes
```


Access Previous Versions

```
git checkout <commit-hash>
```

```
git checkout <tag>
```

```
git restore --source=HEAD~1 file.txt
```

```
git show <commit-hash>:</path/to/file>
```

```
git show HEAD~1:</path/to/file>
```

```
git diff <commit-hash> <commit-hash> -- /path/to/file
```

A new command `git switch` is recommended in place of `checkout`.